CORHUILA
CORPORACIÓN UNIVERSITARIA DEL HUILA
Vigilada Mineducación

INSTITUCIÓN DE EDUCACIÓN SUPERIOR SUJETA A INSPECCIÓN
Y VIGILANCIA POR EL MINISTERIO DE EDUCACIÓN NACIONAL - SNIES 2828

PlanificaIA: agente inteligente para la organización y priorización de tareas académicas

Estudiantes

Dayana Sofía Méndez Perafán

Sneyder Díaz

Docente

José Miguel Llanos Mosquera

Corporación Universitaria del Huila - CORHUILA

Facultad de Ingeniería

Programa de Ingeniería de Sistemas

Electiva Aplicaciones con Inteligencia Artificial

Neiva, Huila

2026

Resumen

PlanificaIA es un agente académico en Python 3 para gestionar tareas y orientar la siguiente acción del estudiante. Combina reglas de vencimiento y prioridad con un modelo de lenguaje consultado mediante Groq. La web permite crear, editar y completar actividades, ver calendario y progreso, y exportar pendientes a PDF. El navegador ofrece dictado y lectura de respuestas. Se probaron consola y sitio público: la regla local seleccionó la entrega más cercana y el chat respondió una consulta real. Las tareas web se guardan en el navegador; la clave API permanece en el servidor.

Palabras clave: agente inteligente, tareas, priorización, educación, Groq.

Abstract

PlanificaIA is an academic agent built in Python 3 to manage assignments and suggest a next action. It combines deadline and priority rules with a language model accessed through Groq. The web interface supports task editing and completion, a calendar, progress tracking, and PDF export of unfinished work. Speech relies on browser features. Command-line and public web checks showed that local rules selected the earliest deadline and the chat answered a real question. Tasks remain in each browser; the API key stays on the server.

Keywords: intelligent agent, tasks, prioritization, education, Groq.

Introducción

La organización del estudio exige coordinar lecturas, trabajos y exposiciones con diferentes fechas y niveles de importancia. Una lista dispersa dificulta identificar las entregas urgentes y medir el avance. El parcial práctico de la electiva solicitó desarrollar un agente inteligente en Python 3, comprobarlo en consola y web y documentar el proceso con evidencias (Corporación Universitaria del Huila, 2026).

Para atender ese problema se desarrolló PlanificaIA. El estudiante registra sus actividades mediante un formulario o conversa con el agente. El sistema muestra lo pendiente y lo completado, ordena los vencimientos y presenta una recomendación explicada. La publicación en Render permite que el docente abra la aplicación desde el enlace sin instalarla.

Justificación

El tema tiene un alcance concreto: cada tarea posee título, fecha límite opcional y prioridad. Estos datos permiten una decisión reproducible. La integración con Groq amplía la interacción al lenguaje natural y permite solicitar o completar información faltante, mientras las reglas locales sostienen la recomendación si el servicio generativo no responde (Groq, s. f.).

El calendario visualiza fechas críticas; el historial conserva el avance y el PDF permite revisar pendientes. La solución es un apoyo individual que utiliza los plazos registrados por el estudiante.

Objetivos

Objetivo general

Desarrollar y publicar un agente en Python 3 que gestione tareas académicas y recomiende cuál atender primero mediante una versión de consola y una interfaz web.

Objetivos específicos

Definir los datos, estados y reglas de prioridad; implementar creación, consulta, modificación y finalización de tareas; integrar un chat con Groq; incorporar calendario, dictado, lectura de voz, historial y exportación PDF; y verificar los flujos principales en consola y en el sitio desplegado.

Tema y alcance del agente

El dominio es la planificación personal de entregas universitarias. La entrada incluye título, fecha, prioridad, materia, notas y duración estimada. La salida puede ser una lista ordenada, una recomendación razonada o una acción sobre una tarea. El agente se limita a la información suministrada y valida los datos antes de operar.

La interfaz guarda el estado en *localStorage*: persiste al recargar el mismo navegador, pero no se sincroniza entre dispositivos (MDN Web Docs, s. f.). La consola usa otro archivo JSON. No hay cuentas, base de datos central ni notificaciones automáticas.

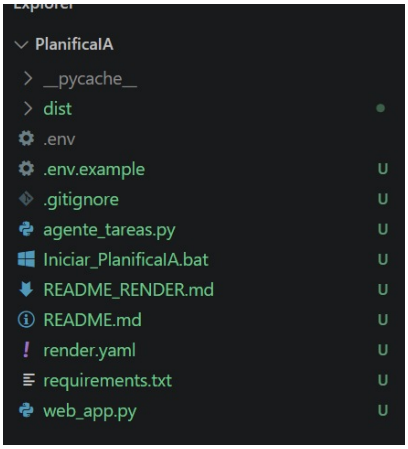
Desarrollo: construcción del agente

Pasos 1 a 3. Se eligió el tema, se acotó el alcance y se definió el modelo de tarea. En *agente_tareas.py* se programaron validaciones, almacenamiento JSON para consola y operaciones de agregar, completar, listar y recomendar. La regla ordena primero las vencidas, luego la fecha más cercana y usa la prioridad como desempate.

Pasos 4 y 5. Se integró Groq para interpretar solicitudes de lenguaje natural y ejecutar herramientas de tareas. Después se creó *web_app.py* con las rutas HTTP de estado, chat, recomendación y salud. La interfaz se implementó en *dist/index.html*.

Figura 1

Archivos del proyecto abiertos en Visual Studio Code.



Nota. Captura del equipo. Se observan el código y los archivos de despliegue.

Desarrollo: interfaz web y conversación

Pasos 6 y 7. Se diseñó una página adaptable con chat, formulario, indicadores, tablero, filtros, calendario e historial. El mensaje escrito o dictado se envía a */api/chat* junto con las tareas pendientes necesarias. El servidor consulta Groq con una clave privada y devuelve la respuesta y las acciones ejecutadas; el navegador actualiza su estado.

La entrada de voz utiliza *SpeechRecognition* cuando el navegador la admite. La respuesta puede leerse con *speechSynthesis* si el usuario activa la casilla correspondiente. El micrófono requiere permiso; por ello también se conserva siempre el campo de texto. El botón Exportar PDF abre la impresión con una vista que contiene solo tareas pendientes.

Figura 2

Chat de la aplicación pública con Groq configurado.



Nota. Captura de https://planificaia.onrender.com/ el 8 de noviembre de 2026. El indicador confirma configuración; la prueba de respuesta real se describe en la página 8.

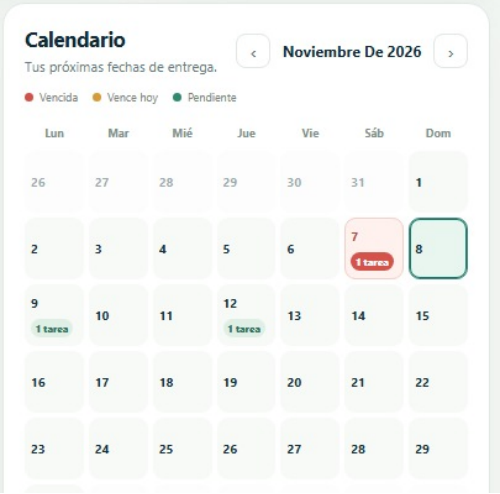
Desarrollo: calendario y seguimiento

Paso 8. Se añadió un calendario mensual que agrupa pendientes por fecha. Una entrega vencida aparece en rojo, una entrega del día en dorado y las próximas en verde. Al seleccionar una fecha se muestra la agenda correspondiente. El tablero permite editar, completar y buscar tareas por texto o materia.

Se cargaron cuatro tareas ficticias en una copia aislada: tres pendientes y una completada. El 8 de noviembre de 2026 se observa una vencida el día 7, dos futuras y 25 % de avance. El sitio público no se modificó.

Figura 3

Calendario con una tarea vencida resaltada en rojo.



Nota. Prueba local con datos ficticios, 8 de noviembre de 2026.

Pruebas funcionales: consola y web

En consola se añadieron “Preparar exposición” (9 de noviembre, alta) y “Leer capítulo” (12 de noviembre, media). *listar* mostró ambas y *recomendar* eligió la exposición por su fecha anterior. Se utilizó una copia aislada.

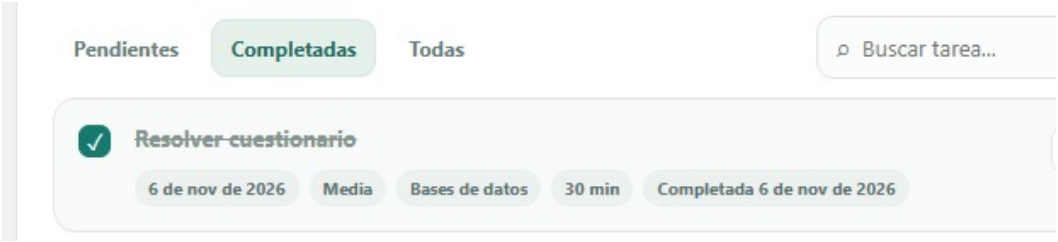
Registro 1. Resumen de salida obtenida en la prueba de consola.

```
Tú > agregar Preparar exposición | 2026-11-09 | alta
Agregada [182a4ee9]: Preparar exposición
Tú > agregar Leer capítulo | 2026-11-12 | media
Agregada [01d389fd]: Leer capítulo
Tú > recomendar
Agente (Reglas locales) > Te recomiendo empezar por Preparar exposición.
```

La web pública respondió una consulta real sobre sus funciones. En la copia local, una tarea completada pasó al historial y aumentó el progreso.

Figura 4

Historial de una tarea completada.



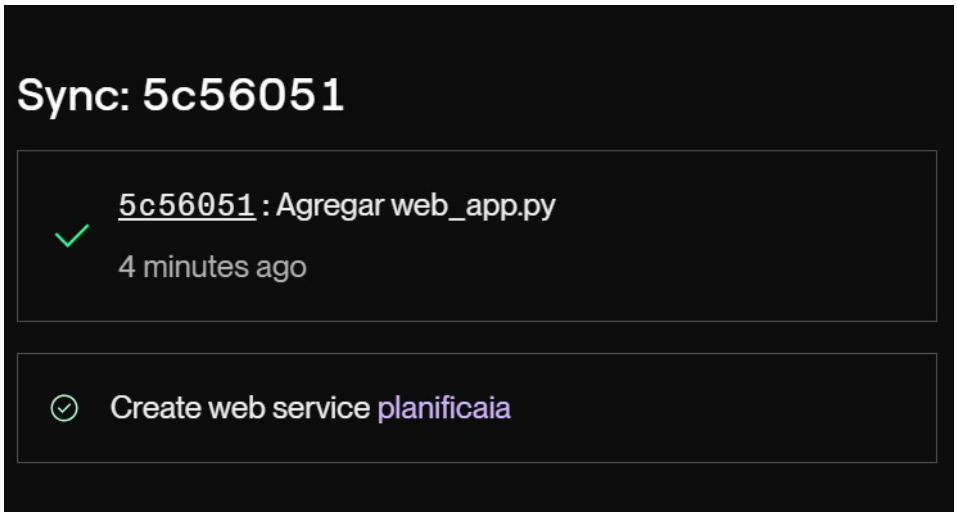
Nota. Prueba local con datos ficticios; vista Completadas.

Resultados obtenidos y despliegue

Paso 9. El código se publicó en GitHub y Render se configuró con *render.yaml*. El Blueprint instala dependencias, inicia *web_app.py* y verifica */health*. La clave *GROQ_API_KEY* quedó como variable privada del servicio (Render, s. f.).

Figura 5

Sincronización del Blueprint y creación del servicio web planificaia.



Nota. Captura del panel de Render proporcionada por el grupo. La URL pública se comprobó por separado.

La aplicación está disponible en https://planificaia.onrender.com/. Se comprobó el chat, el calendario y la ruta */health*. El PDF de pendientes se obtiene desde la impresión del navegador. La voz requiere permiso y compatibilidad; no se documenta una prueba física de micrófono. Las tareas web no se sincronizan entre usuarios.

Conclusiones

Se construyó un agente funcional en consola y web para registrar, completar y priorizar tareas académicas. La prueba de consola confirmó el orden por fecha y la página pública respondió una consulta mediante Groq. El calendario, el historial y la exportación de pendientes permiten demostrar el funcionamiento completo durante la sustentación.

Recomendaciones

Para la demostración conviene preparar tareas ficticias en el mismo navegador y abrir el sitio antes de la exposición. Como evolución se propone una base de datos con cuentas y sincronización entre dispositivos, además de pruebas automatizadas de voz en navegadores compatibles.

Referencias

Corporación Universitaria del Huila. (2026). *Parcial práctico tercer corte: Aplicaciones con Inteligencia Artificial* [Guía de evaluación].

Groq. (s. f.). *Text generation*. GroqDocs. https://console.groq.com/docs/text-chat

MDN Web Docs. (s. f.). *Window: localStorage property*. https://developer.mozilla.org/en-US/docs/Web/API/Window/localStorage

Méndez Perafán, D. S., & Díaz, S. (2026). *PlanificaIA* [Código fuente]. GitHub. https://github.com/DayanaMendess/PlanificaIA

Render. (s. f.). *Blueprint YAML reference*. https://render.com/docs/blueprint-spec